



UAI301 Object Oriented Programming

Lab Assignment 4

Name: Amritanshu Mishra

Roll No.: 1025250008

Sub-Group: 2X11

Subject Code: UAI301

Subject Name: Object Oriented Programming

Question 1 Consider the class MyClass with x = 5. Write a Python program to print MyClass, and access and print the value of x using MyClass.x.

```
In [1]: class MyClass:
        x = 5

        print(MyClass)
        print(MyClass.x)

<class '__main__.MyClass'>
5
```

Question 2 Write a Python program using a Student class with class variables college and country, and instance variables name and roll. Create two student objects and display their details.

```
In [2]: class Student:
        college = "TIET"
        country = "India"

        def __init__(self, name, roll):
            self.name = name
            self.roll = roll

        s1 = Student("Amritanshu", 1025250008)
        s2 = Student("Rahul", 1025250034)

        print("Student 1:", s1.name, s1.roll, s1.college, s1.country)
        print("Student 2:", s2.name, s2.roll, s2.college, s2.country)

Student 1: Amritanshu 1025250008 TIET India
Student 2: Rahul 1025250034 TIET India
```

Question 3 Write a Python program that demonstrates that a class itself is an object. Print Student, print type(Student), and print Student.college without

creating any object.

```
In [3]: class Student:
        college = "TIET"

        print(Student)
        print(type(Student))
        print(Student.college)

<class '__main__.Student'>
<class 'type'>
TIET
```

Question 4 Write a Python program to access the class variable `app_version` of an Instagram class using the class name, a temporary object, and a named object.

```
In [4]: class Instagram:
        app_version = "2.5.1"

        print("Using class name:", Instagram.app_version)
        print("Using temporary object:", Instagram().app_version)

        insta = Instagram()
        print("Using named object:", insta.app_version)

Using class name: 2.5.1
Using temporary object: 2.5.1
Using named object: 2.5.1
```

Question 5 Create a Netflix class with class variable `subscription = 'Premium'`. Demonstrate accessing it using the class name, a temporary object, and a named object.

```
In [5]: class Netflix:
        subscription = "Premium"

        print("Using class name:", Netflix.subscription)
        print("Using temporary object:", Netflix().subscription)

        nf = Netflix()
        print("Using named object:", nf.subscription)

Using class name: Premium
Using temporary object: Premium
Using named object: Premium
```

Question 6 Develop an Employee Management System using a class variable `company` and instance variables `name` and `salary`. Modify `emp1`'s salary and the class variable `company`, then observe which changes affect one object vs all objects.

```
In [6]: class Employee:
        company = "Google"

        def __init__(self, name, salary):
            self.name = name
            self.salary = salary

emp1 = Employee("Aman", 50000)
emp2 = Employee("Riya", 60000)

print("Initial Details")
print(emp1.company, emp1.name, emp1.salary)
print(emp2.company, emp2.name, emp2.salary)

emp1.salary = 55000
Employee.company = "Alphabet"

print("\nAfter Modification")
print(emp1.company, emp1.name, emp1.salary)
print(emp2.company, emp2.name, emp2.salary)

print("\nObservation: Changing emp1.salary only affected emp1 (instance v
print("while changing Employee.company affected both objects (class varia
```

```
Initial Details
Google Aman 50000
Google Riya 60000
```

```
After Modification
Alphabet Aman 55000
Alphabet Riya 60000
```

Observation: Changing emp1.salary only affected emp1 (instance variable), while changing Employee.company affected both objects (class variable).

Question 7 Write a Python program using the Student class to demonstrate print(Student), type(Student), creating two objects, printing their types, and accessing class/instance variables.

```
In [7]: class Student:
        college = "TIET"

        def __init__(self, name, roll):
            self.name = name
            self.roll = roll

print(Student)
print(type(Student))

s1 = Student("Amritanshu", 1)
s2 = Student("Rahul", 2)

print(type(s1))
```

```
print(type(s2))

print(Student.college)
print(s1.name, s1.roll)
print(s2.name, s2.roll)

print("\nprint(Student) shows the class object itself,")
print("while print(s1) shows an instance object of that class.")
```

```
<class '__main__.Student'>
<class 'type'>
<class '__main__.Student'>
<class '__main__.Student'>
TIET
Amritanshu 1
Rahul 2
```

print(Student) shows the class object itself,
while print(s1) shows an instance object of that class.

Question 8 Write a Python program using an Election class with constructor def **init**(self, state, voters=None, leader=None) to create objects for Punjab, Uttar Pradesh, and Bihar, and observe the role of None as a default argument.

```
In [8]: class Election:
        def __init__(self, state, voters=None, leader=None):
            self.state = state
            self.voters = voters
            self.leader = leader

punjab = Election("Punjab", 20_000_000)
up = Election("Uttar Pradesh", 150_000_000)
bihar = Election("Bihar", 20000, "Aman")

print("Voters of Punjab:", punjab.voters)
print("Voters of Uttar Pradesh:", up.voters)
print("Leader of Bihar:", bihar.leader)

bihar.leader = "Nitish"
print("Modified Leader of Bihar:", bihar.leader)

print("\nNo error occurs for Punjab and Uttar Pradesh because 'leader' has a default value of None,
so it does not need to be supplied. None acts as a safe placeholder default argument.")
```

```
Voters of Punjab: 20000000
Voters of Uttar Pradesh: 150000000
Leader of Bihar: Aman
Modified Leader of Bihar: Nitish
```

No error occurs for Punjab and Uttar Pradesh because 'leader' has a default value of None,
so it does not need to be supplied. None acts as a safe placeholder default argument.

Question 9 Modify the Election constructor to make voters and leader mandatory arguments, attempt to create an object without leader, and observe the resulting error compared to using default arguments.

```
In [9]: class Election:
        def __init__(self, state, voters, leader):
            self.state = state
            self.voters = voters
            self.leader = leader

        try:
            punjab = Election("Punjab", 20000000)
        except TypeError as e:
            print("Error:", e)

        print("\nThis error occurs because 'leader' is a mandatory argument with
        print("so it must be supplied when creating the object.")
        print("When voters=None and leader=None are used as defaults, the argumen
        print("and the object can be created without providing them.")
```

Error: Election.__init__() missing 1 required positional argument: 'leader'

This error occurs because 'leader' is a mandatory argument with no default value,
so it must be supplied when creating the object.
When voters=None and leader=None are used as defaults, the arguments become optional,
and the object can be created without providing them.

Question 10 Create a class Patient with attributes Name, Age, and Blood Group. Initialize using a constructor and display the details using a member function.

```
In [10]: class Patient:
        def __init__(self, name, age, blood_group):
            self.name = name
            self.age = age
            self.blood_group = blood_group

        def display(self):
            print("Name:", self.name)
            print("Age:", self.age)
            print("Blood Group:", self.blood_group)

        p1 = Patient("Rahul Sharma", 45, "B+")
        p1.display()
```

Name: Rahul Sharma
Age: 45
Blood Group: B+

Question 11 Create a class Student with Name and Marks in four subjects. Create a method to calculate and display Average Marks and Grade (Grade A if average ≥ 75 , else Grade B).

```
In [11... class Student:
    def __init__(self, name, m1, m2, m3, m4):
        self.name = name
        self.m1 = m1
        self.m2 = m2
        self.m3 = m3
        self.m4 = m4

    def result(self):
        average = (self.m1 + self.m2 + self.m3 + self.m4) / 4
        grade = "A" if average >= 75 else "B"
        print("Name:", self.name)
        print("Average Marks:", average)
        print("Grade:", grade)

s1 = Student("Amritanshu", 80, 78, 85, 90)
s1.result()
```

```
Name: Amritanshu
Average Marks: 83.25
Grade: A
```

Question 12 Create a class BankAccount with Account Holder Name, Account Number, and Balance. Create methods Deposit Amount, Withdraw Amount, and Display Balance.

```
In [12... class BankAccount:
    def __init__(self, holder_name, acc_number, balance=0):
        self.holder_name = holder_name
        self.acc_number = acc_number
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount
        print(f"Deposited {amount}. New Balance: {self.balance}")

    def withdraw(self, amount):
        if amount <= self.balance:
            self.balance -= amount
            print(f"Withdrew {amount}. New Balance: {self.balance}")
        else:
            print("Insufficient funds")

    def display_balance(self):
        print("Account Holder:", self.holder_name)
        print("Account Number:", self.acc_number)
        print("Balance:", self.balance)
```

```
acc = BankAccount("Rahul Sharma", "AC1234", 5000)
acc.deposit(2000)
acc.withdraw(3000)
acc.display_balance()
```

Deposited 2000. New Balance: 7000
 Withdrew 3000. New Balance: 4000
 Account Holder: Rahul Sharma
 Account Number: AC1234
 Balance: 4000

Question 13 Create an InventoryItem class with attributes item_name, sku, price, and stock_quantity. Implement methods to display item details, purchase item (reduce stock), and restock item (increase stock).

In [13...

```
class InventoryItem:
    def __init__(self, item_name, sku, price, stock_quantity):
        self.item_name = item_name
        self.sku = sku
        self.price = price
        self.stock_quantity = stock_quantity

    def display_details(self):
        print(f"Item: {self.item_name}, SKU: {self.sku}, Price: {self.price}, Stock: {self.stock_quantity}")

    def purchase_item(self, quantity):
        if quantity <= self.stock_quantity:
            self.stock_quantity -= quantity
            print(f"Purchased {quantity} units. Remaining stock: {self.stock_quantity}")
        else:
            print("Not enough stock available")

    def restock_item(self, quantity):
        self.stock_quantity += quantity
        print(f"Restocked {quantity} units. Current stock: {self.stock_quantity}")

item = InventoryItem("Wireless Mouse", "SKU001", 799, 50)
item.display_details()
item.purchase_item(10)
item.restock_item(20)
item.display_details()
```

Item: Wireless Mouse, SKU: SKU001, Price: 799, Stock: 50
 Purchased 10 units. Remaining stock: 40
 Restocked 20 units. Current stock: 60
 Item: Wireless Mouse, SKU: SKU001, Price: 799, Stock: 60

Question 14 Create a FitnessTracker class with attributes user_name, daily_step_goal, and steps_walked. Implement methods to calculate progress percentage and display whether the goal was achieved.

In [14...

```
class FitnessTracker:
    def __init__(self, user_name, daily_step_goal, steps_walked):
```

```

        self.user_name = user_name
        self.daily_step_goal = daily_step_goal
        self.steps_walked = steps_walked

    def progress_percentage(self):
        percentage = (self.steps_walked / self.daily_step_goal) * 100
        print(f"{self.user_name}'s progress: {percentage:.2f}%")
        return percentage

    def goal_status(self):
        if self.steps_walked >= self.daily_step_goal:
            print(f"{self.user_name} has achieved their fitness goal for the day.")
        else:
            print(f"{self.user_name} has not yet achieved their fitness goal for the day.")

tracker = FitnessTracker("Aman", 10000, 8500)
tracker.progress_percentage()
tracker.goal_status()

```

Aman's progress: 85.00%

Aman has not yet achieved their fitness goal for the day.

Question 15 Case Study: Smart Home Energy Management System. Create a SmartAppliance class with appliance_name, room_location, power_rating_watts, and is_on. Implement turn_on(), turn_off(), and display_status(). Create two objects and confirm their states are managed independently.

```

In [15... class SmartAppliance:
    def __init__(self, appliance_name, room_location, power_rating_watts, is_on):
        self.appliance_name = appliance_name
        self.room_location = room_location
        self.power_rating_watts = power_rating_watts
        self.is_on = is_on

    def turn_on(self):
        self.is_on = True
        print(f"{self.appliance_name} in {self.room_location} is now ON.")

    def turn_off(self):
        self.is_on = False
        print(f"{self.appliance_name} in {self.room_location} is now OFF.")

    def display_status(self):
        state = "ON" if self.is_on else "OFF"
        print(f"Appliance: {self.appliance_name}, Room: {self.room_location}, Power Rating: {self.power_rating_watts}W, Status: {state}")

ac_unit = SmartAppliance("Air Conditioner", "Living Room", 1500)
smart_light = SmartAppliance("Smart Light", "Bedroom", 15)

ac_unit.turn_on()

```



```
ac_unit.display_status()
smart_light.display_status()

print("\nBoth objects manage their 'is_on' state independently.")
```

Air Conditioner in Living Room is now ON.

Appliance: Air Conditioner, Room: Living Room, Power Rating: 1500W, Status: ON

Appliance: Smart Light, Room: Bedroom, Power Rating: 15W, Status: OFF

Both objects manage their 'is_on' state independently.

Question 16 Case Study: A smart city Smart Energy Board wants a program to monitor substations using an EnergySubstation class with class variables city_name and total_city_power_drawn, and instance methods add_load() and display_status(), demonstrating overload handling and citywide tracking.

```
In [16... class EnergySubstation:
    city_name = "Neo Metro"
    total_city_power_drawn = 0.0

    def __init__(self, substation_id, max_capacity_kw):
        self.substation_id = substation_id
        self.max_capacity_kw = max_capacity_kw
        self.current_load_kw = 0.0

    def add_load(self, amount):
        if self.current_load_kw + amount <= self.max_capacity_kw:
            self.current_load_kw += amount
            EnergySubstation.total_city_power_drawn += amount
            print(f"{self.substation_id}: Added {amount} kW load. "
                  f"Current load: {self.current_load_kw}/{self.max_capacity_kw}")
        else:
            print(f"{self.substation_id}: Overload warning! Cannot add {amount} kW")

    def display_status(self):
        print(f"Substation: {self.substation_id}, "
              f"Load: {self.current_load_kw}/{self.max_capacity_kw} kW, "
              f"City: {EnergySubstation.city_name}")

sub_1 = EnergySubstation("SUB_NORTH", 500.0)
sub_2 = EnergySubstation("SUB_SOUTH", 300.0)

sub_1.add_load(350.0)
sub_2.add_load(200.0)
sub_2.add_load(150.0)

sub_2.max_capacity_kw = 400.0
sub_2.add_load(100.0)

sub_1.display_status()
sub_2.display_status()
```

```
print("Total citywide power drawn:", EnergySubstation.total_city_power_dr
```

```
SUB_NORTH: Added 350.0 kW load. Current load: 350.0/500.0 kW.  
SUB_SOUTH: Added 200.0 kW load. Current load: 200.0/300.0 kW.  
SUB_SOUTH: Overload warning! Cannot add 150.0 kW load.  
SUB_SOUTH: Added 100.0 kW load. Current load: 300.0/400.0 kW.  
Substation: SUB_NORTH, Load: 350.0/500.0 kW, City: Neo Metro  
Substation: SUB_SOUTH, Load: 300.0/400.0 kW, City: Neo Metro  
Total citywide power drawn: 650.0
```